

Multi-VPC Networking with AWS PrivateLink

NAT Gateway Edition — Project Write-Up

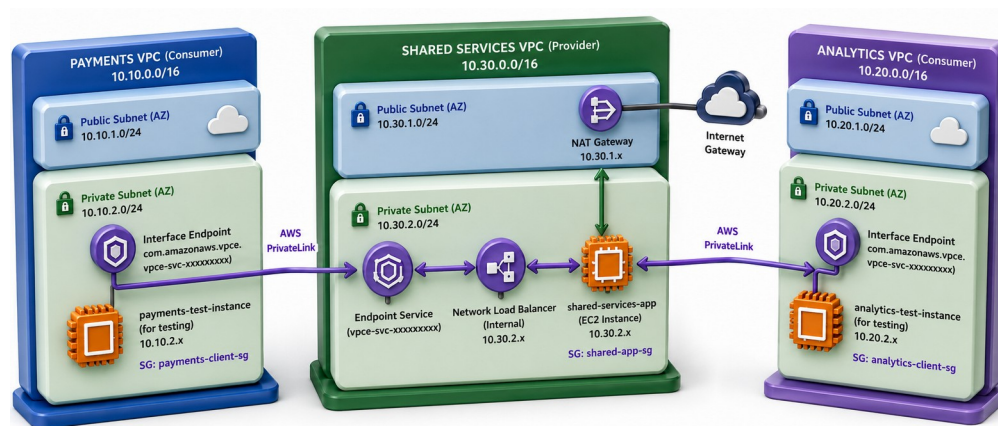
Project name: Multi-VPC Networking with AWS PrivateLink

Your name / email: Dwayne Cowart · dcowart87@gmail.com

Date completed: July 5, 2026

Introducing This Project

In this project, I built a private, multi-VPC network for a fictional fintech company with three isolated systems — Payments, Analytics, and Shared Services — connected exclusively through AWS PrivateLink. The Shared Services VPC hosts an internal web app that has no inbound path from the internet at all; the only way in is through approved PrivateLink connections from Payments and Analytics. I used a NAT Gateway (rather than a custom AMI pipeline) to give the private instance outbound-only internet access for installing software, then verified end-to-end that the service was reachable only from approved VPCs and unreachable from the public internet.



Architecture overview — Payments and Analytics VPCs (PrivateLink consumers) connecting to the Shared Services VPC (PrivateLink provider), which reaches the internet only outbound through its NAT Gateway.

Tools and Concepts

I used Amazon VPC, Subnets, Route Tables, and an Internet Gateway to build three isolated networks; a NAT Gateway to give the Shared Services private subnet outbound-only internet access; Amazon EC2 to host the internal app; Security Groups for fine-grained network access control; an internal Network Load Balancer (NLB) to front the app; and AWS PrivateLink — specifically a VPC Endpoint Service on the provider side and Interface Endpoints on the consumer side — to connect the VPCs privately. I also used an EC2 Instance Connect Endpoint for browser-based admin access to instances that have no public IP.

Key concepts reinforced by this project: the difference between a route table controlling whether a path exists versus a security group controlling whether traffic is allowed; how NAT Gateways provide outbound-only internet access without exposing anything inbound; and how Network Load Balancer client IP preservation changes which source IPs a downstream security group actually needs to allow.

Project Set Up

Three VPCs and a NAT Gateway

I created three VPCs using the VPC wizard's "VPC and more" option, each with one public and one private subnet in a single Availability Zone:

- payments-vpc — 10.10.0.0/16 (public 10.10.1.0/24, private 10.10.2.0/24)
- analytics-vpc — 10.20.0.0/16 (public 10.20.1.0/24, private 10.20.2.0/24)
- shared-services-vpc — 10.30.0.0/16 (public 10.30.1.0/24, private 10.30.2.0/24)

Payments and Analytics never need general internet access — they only ever talk out via PrivateLink — so I set NAT gateways to None for both. Shared Services needed outbound internet access for installing and patching software on its private instance, so I set NAT gateways to In 1 AZ, which automatically provisioned a NAT Gateway in the public subnet and wired the private route table's 0.0.0.0/0 route to it.

NAT gateways (\$) - *updated* [Info](#)
NAT gateway allows private resources to access the internet from any availability zone within a VPC, providing a single managed internet exit point for the entire region. Additional charges apply.

None | Regional - new | Zonal

Introducing regional NAT gateway ×

AWS now offers a multi-AZ NAT Gateway, eliminating the need for separate NAT Gateways across availability zones.

VPC endpoints [Info](#)
Endpoints can help reduce NAT gateway charges and improve security by accessing S3 directly from the VPC. By default, full access policy is used. You can customize this policy at any time.

None | **S3 Gateway**

DNS options [Info](#)
☒ Enable DNS hostnames
☒ Enable DNS resolution

► Additional tags

NAT gateway setting for Payments and Analytics VPCs — set to None.

NAT gateways (\$) [Info](#)

Choose the number of Availability Zones (AZs) in which to create NAT gateways.
Note that there is a charge for each NAT gateway

In 1 AZ | 1 per AZ

VPC endpoints [Info](#)

Endpoints can help reduce NAT gateway charges and improve security by accessing S3 directly from the VPC. By default, full access policy is used. You can customize this policy at any time.

None | **S3 Gateway**

DNS options [Info](#)

- ☒ Enable DNS hostnames
- ☒ Enable DNS resolution

NAT gateway setting for Shared Services VPC — set to In 1 AZ, with DNS hostnames and resolution both enabled.

Security Groups

I created three main security groups scoped tightly to their purpose:

- shared-app-sg (in shared-services-vpc) — allows inbound TCP 80 from the Shared Services VPC CIDR (covers NLB health checks), plus later an SSH rule for the EC2 Instance Connect Endpoint and additional port-80 rules for Payments and Analytics (see Troubleshooting Notes).
- payments-client-sg (in payments-vpc) — allows inbound TCP 80 from within the Payments VPC CIDR; attached to the Payments Interface Endpoint.
- analytics-client-sg (in analytics-vpc) — allows inbound TCP 80 from within the Analytics VPC CIDR; attached to the Analytics Interface Endpoint.

sg-0de0c8d93ab7a4048 - shared-app-sg

Details

Security group name
shared-app-sg

Security group ID
sg-0de0c8d93ab7a4048

Description
Shared Services app - PrivateLink traffic on port 80

VPC ID
vpc-0e7b5f00312ca76da [L](#)

Owner
783149339342

Inbound rules count
1 Permission entry

Outbound rules count
1 Permission entry

Inbound rules | Outbound rules | Sharing | VPC associations | Related resources - *new* | Tags

Inbound rules (1)

☐ **Name**

☐ **Security group rule ID**

☐ **IP version**

☐ **Type**

☐ **Protocol**

☐ **Port range**

☐ **Source**

☐ -

sg-03c7ee687db010976

IPv4

HTTP

TCP

80

10.30.0.0/16

shared-app-sg — inbound rule allowing HTTP/80 from 10.30.0.0/16 (Shared Services VPC CIDR).

sg-038744a668094445a - payments-client-sg

Details

Security group name
payments-client-sg

Security group ID
sg-038744a668094445a

Description
Interface Endpoint SG for Payments - PrivateLink client

VPC ID
vpc-0686611d8c7ea7c2e [L](#)

Owner
783149339342

Inbound rules count
1 Permission entry

Outbound rules count
1 Permission entry

Inbound rules | Outbound rules | Sharing | VPC associations | Related resources - *new* | Tags

Inbound rules (1)

☐ **Name**

☐ **Security group rule ID**

☐ **IP version**

☐ **Type**

☐ **Protocol**

☐ **Port range**

☐ **Source**

☐ -

sg-058dcbf5957b93ac2

IPv4

HTTP

TCP

80

10.10.0.0/16

payments-client-sg — inbound rule allowing HTTP/80 from 10.10.0.0/16 (Payments VPC CIDR).

sg-0385bacd556dd482f - analytics-client-sg Actions

Details

Security group name
analytics-client-sg
Security group ID
sg-0385bacd556dd482f
Description
Interface Endpoint SG for Analytics - PrivateLink client
VPC ID
vpc-0762694b4aafae426

Owner
783149339342
Inbound rules count
1 Permission entry
Outbound rules count
1 Permission entry

Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags

Inbound rules (1)

Search

	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
☐	-	sgr-095d6e68f0c3cbd92	IPv4	HTTP	TCP	80	10.20.0.0/16

analytics-client-sg — inbound rule allowing HTTP/80 from 10.20.0.0/16 (Analytics VPC CIDR).

Private EC2 Instance

I launched a single EC2 instance, shared-services-app, in the private subnet of shared-services-vpc with no public IP and the shared-app-sg security group attached. Because the private subnet now has outbound internet access through the NAT Gateway, I installed the app directly with ordinary User Data — no builder instance or custom AMI was needed:

```
#!/bin/bash
yum update -y
yum install -y httpd

cat > /var/www/html/index.html << 'EOF'
<h1>PrivateLink Internal Service</h1>
<p>This internal API is only reachable from approved VPCs via AWS PrivateLink.</p>
EOF

systemctl enable httpd
systemctl start httpd
```

Since the instance has no public IP and there's no bastion host, I connected to it for verification using an EC2 Instance Connect Endpoint (EICE) rather than a public IP or a jump box.

Instances (1/2) Info Last updated 41 minutes ago Connect Instance state Actions Launch instances

Saved filter sets Choose filter set

☐	Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...
☐	payments-test-instance	i-0ee19169baaec3200	Running	t3.micro	2/2 checks passed	us-east-1a	-	-
☒	shared-services-app	i-067f2399b8023d61e	Running	t3.micro	3/3 checks passed	us-east-1a	-	-

shared-services-app and payments-test-instance both running with all status checks passed.

Admin Access Setup — EC2 Instance Connect Endpoint

To reach instances with no public IP for verification, I created an EC2 Instance Connect Endpoint in the private subnet of shared-services-vpc. This gave me a browser-based terminal session tunneled directly into the private instance, with no public IP, bastion host, or SSH key exchange over the internet required.

Create endpoint info

Create the type of VPC endpoint that supports the service, service network or resource to which you want to connect.

Endpoint settings info

Specify a name and select the type of endpoint.

Name tag - optional
 Create a tag with a key of 'Name' and a value that you specify. Tags help you find and manage your endpoint.

Type info
 Select a category

☐ AWS services
 Connect to services provided by Amazon with an interface endpoint, or a Gateway endpoint

☐ PrivateLink Ready partner services
 Connect to SaaS services which have AWS Service Ready designation with an interface endpoint. Uses AWS PrivateLink

☐ AWS Marketplace services
 Connect to SaaS services that you have purchased through AWS Marketplace with an interface endpoint.

☒ EC2 Instance Connect Endpoint
 An elastic network interface that allows you to connect to resources in a private subnet

☐ Resources
 Connect to resources like Amazon Relational Database Services (RDS) with a Resource endpoint. Uses AWS PrivateLink

☐ Service networks
 Connect to VPC Lattice service networks with a Service network endpoint. Uses AWS PrivateLink

☐ Endpoint services that use NLBs and GWLBs
 Find services shared with you by service name. Connect to a Network LoadBalancer (NLB) service with an interface endpoint or to a Gateway LoadBalancer (GWLB) service with a Gateway Load Balancer endpoint

Network settings info

Select the VPC in which to create the endpoint

VPC
 Create the VPC endpoint in the VPC in the same AWS Region from which you will access a resource.
 🔄

Additional settings

Security groups (1/8) info

Group ID	Group name	VPC ID	Description
☐ sg-079a20807d6c5c35e	default	vpc-0dfe4f7262894d5da	default VPC security group
☐ sg-0a876f6646409d543	default	vpc-0762694b4aafae436	default VPC security group
☐ sg-0385bae055646882f	analytics-client-sg	vpc-0762694b4aafae436	Interface Endpoint SG for Analytics - PrivateLink client
☐ sg-021f82a851752a2f	launch-wizard-1	vpc-0dfe4f7262894d5da	launch-wizard-1 created 2026-06-08T19:27:59.015Z
☒ sg-0dc0c895ab7a4048	shared-app-sg	vpc-0e7b5f00312ca76da	Shared Services app - PrivateLink traffic on port 80
☐ sg-0f6462467f9f852b1	default	vpc-0e7b5f00312ca76da	default VPC security group
☐ sg-039744a6d8094445a	payments-client-sg	vpc-0686611db87ea7c2e	Interface Endpoint SG for Payments - PrivateLink client
☐ sg-0ee235c34c9668fce	default	vpc-0686611db87ea7c2e	default VPC security group

✕

Subnet info

Select the Subnet in which to create the endpoint

Subnet
 Select the subnets in which to create the endpoint.
 🔄

IP address type

☒ IPv4
☐ IPv6
☐ Dualstack

Tags

Key Value - optional ✕ Remove

Add new tag
 You can add 49 more tags.

Cancel Create endpoint

Creating the shared-services-eice EC2 Instance Connect Endpoint in the Shared Services private subnet.

One thing I had to add afterward: shared-app-sg only allowed inbound port 80 at first, so I added an inbound SSH (22) rule from the Shared Services VPC CIDR (10.30.0.0/16) — this is what actually lets the endpoint's network interface reach the instance over SSH.

Once connected, I verified the install worked before moving on:

```
[ec2-user@ip-10-30-2-xx ~]$ systemctl status httpd
● httpd.service - The Apache HTTP Server
   Active: active (running)
[ec2-user@ip-10-30-2-xx ~]$ curl localhost
<h1>PrivateLink Internal Service</h1>
```

PrivateLink Provider & Consumer Setup

Provider side — Target Group, NLB, and Endpoint Service

I created a Target Group (shared-services-tg, protocol TCP, port 80) in shared-services-vpc and registered shared-services-app as its target. I then created an internal Network Load Balancer, shared-services-nlb, scoped to the private subnet, with a TCP/80 listener forwarding to the target group.

Create Network Load Balancer

The Network Load Balancer distributes incoming TCP and UDP traffic across multiple targets such as Amazon EC2 instances, microservices, and containers. When the load balancer receives a connection request, it selects a target based on the protocol and port that are specified in the listener configuration, and the routing rule specified in the default action.

How Network Load Balancers work

Basic configuration

Load balancer name
Name must be unique within your AWS account and can't be changed after the load balancer is created.
shared-services-nlb

Scheme
Scheme can't be changed after the load balancer is created.
☐ Internet-facing
☒ Internal

Load balancer IP address type
Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types.
☒ IPv4
☐ Dualstack

Network mapping

VPC
The load balancer routes traffic to targets in the selected subnets, and in accordance with your IP address settings.
vpc-0e7b5f00312ca76da (shared-services-vpc)

Availability Zones and subnets
Select one or more Availability Zones and corresponding subnets. Enabling multiple Availability Zones increases the fault tolerance of your applications. The load balancer routes traffic to targets in the selected Availability Zones only. Availability Zones that are not supported by the load balancer or the VPC are not available for selection.
☒ us-east-1a (use1-az1)

Creating shared-services-nlb — Scheme set to Internal, in shared-services-vpc's private subnet.

Target groups (1/1)

Name	ARN	Port	Protocol	Target type	Load balancer	VPC ID
shared-services-tg	arn:aws:elasticloadbalancing:us-east-1:123456789012:targetgroup/shared-services-tg:12345678901234567890	80	TCP	Instance	shared-services-nlb	vpc-0e7b5f00312ca76da

Target group: shared-services-tg

Registered targets (1)

Instance ID	Name	Port	Zone	Health status	Health status ...	Administrati...	Override det...	Launch time
i-067f2399b80...	shared-services...	80	us-east-1a (use1-az1)	Healthy		No override	No override is c...	July 5, 2026, 1...

shared-services-tg's registered target showing a Healthy status — confirms the NLB can actually route traffic to the instance.

With the NLB in place, I created a VPC Endpoint Service pointing at it, with "Require acceptance for endpoint connections" left enabled — this is the PrivateLink provider (server) side.

Endpoint services (1)

Name	Service ID	Types	Service name	State	Availability Zones	Acc...
shared-internal-service	vpce-svc-0e4aa4cebb14984cb	Interface	com.amazonaws.vpce-us-east-1.vpce-sv...	Available	use1-az1 (us-east-1a)	Yes

The shared-internal-service Endpoint Service showing State: Available, with its generated service name visible.

Consumer side — Interface Endpoints

In both payments-vpc and analytics-vpc, I created an Interface Endpoint of type "Endpoint services that use NLBs and GWLBs," pasted in the generated service name, and attached each VPC's respective client security group.

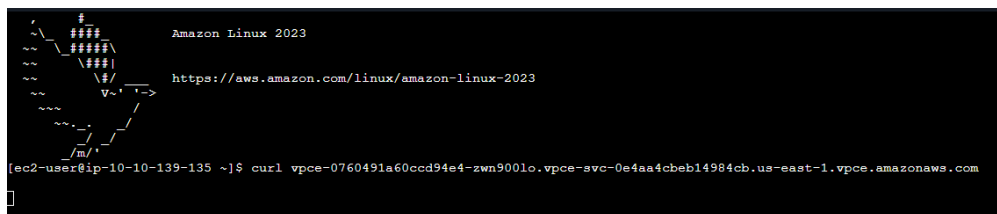
Both connections came up as Pending acceptance, which I then approved from the Endpoint Service's Endpoint connections tab.

Troubleshooting Notes

The most valuable lesson from this build came from testing connectivity end to end — the app was unreachable even from an approved VPC at first, and figuring out why taught me something specific to NLB + PrivateLink + instance targets.

Symptom: curl hung with no response

After launching a test instance in payments-vpc's private subnet and curling the Interface Endpoint's private DNS name, the request just hung indefinitely instead of returning the app's HTML or failing quickly.



```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

[ec2-user@ip-10-10-139-135 ~]$ curl vpce-0760491a60ccd94e4-zwn9001o.vpce-svc-0e4aa4cbeb14984cb.us-east-1.vpce.amazonaws.com
```

curl to the Interface Endpoint's DNS name hangs with no response.

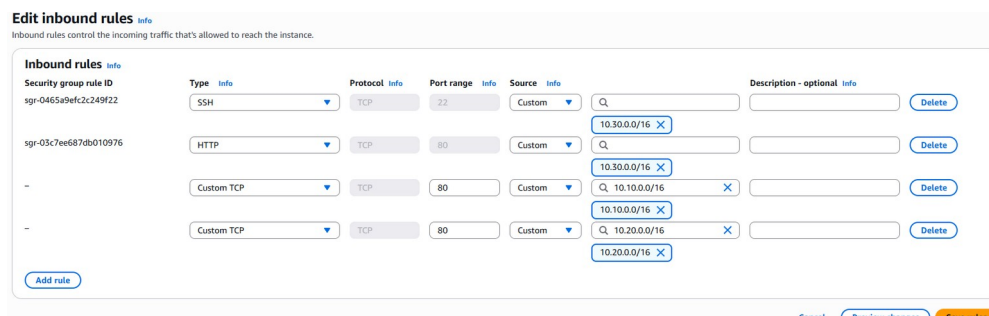
Diagnosis: NLB client IP preservation

The original design assumed that once traffic crossed into shared-services-vpc via PrivateLink and the NLB, it would appear to arrive from somewhere inside 10.30.0.0/16 — so shared-app-sg was scoped to allow port 80 only from that CIDR. In reality, Network Load Balancers enable client IP preservation by default whenever the target type is Instance (as opposed to IP). With this enabled, the NLB does not perform source NAT — the packet that reaches shared-services-app still carries the original client's IP address, in this case an address from payments-test-instance's subnet (10.10.2.0/24), not from 10.30.0.0/16.

Security groups are stateful firewalls that drop denied traffic silently — there is no rejection sent back and no error shown anywhere. That silent drop is exactly what produced the hang: from curl's perspective, a silently dropped SYN packet is indistinguishable from a slow server, so it just waits until it times out. A fast "connection refused" would instead point to something above the network layer (like nothing listening on the port); a hang points to a network- or security-group-level drop.

Fix: add rules for the real client CIDRs

I added inbound rules to shared-app-sg for TCP port 80 from 10.10.0.0/16 (Payments) and 10.20.0.0/16 (Analytics), in addition to the existing 10.30.0.0/16 rule (which still correctly covers the NLB's own health checks, originating from inside its own subnet).



Edit inbound rules Info
Inbound rules control the incoming traffic that's allowed to reach the instance.

Type	Protocol	Port range	Source	Description - optional	
SSH	TCP	22	Custom	10.30.0.0/16	Delete
HTTP	TCP	80	Custom	10.30.0.0/16	Delete
Custom TCP	TCP	80	Custom	10.10.0.0/16	Delete
Custom TCP	TCP	80	Custom	10.20.0.0/16	Delete

[Add rule](#) [Cancel](#) [Preview changes](#) [Save rules](#)

shared-app-sg updated with inbound rules for SSH plus HTTP from 10.30.0.0/16, 10.10.0.0/16, and 10.20.0.0/16.

After saving these rules, the curl request from payments-test-instance succeeded immediately.

This is worth flagging as a gotcha specific to this exact combination — NLB, PrivateLink, and Instance-type targets. It wouldn't have appeared with IP-type targets, or if client IP preservation had been explicitly disabled on the target group. There's a real design tradeoff here too: preserving the client's real IP is often desirable (for logging or IP-based access control at the app layer), but it means the security group has to account for every VPC that might legitimately connect — not just the VPC the load balancer itself sits in.

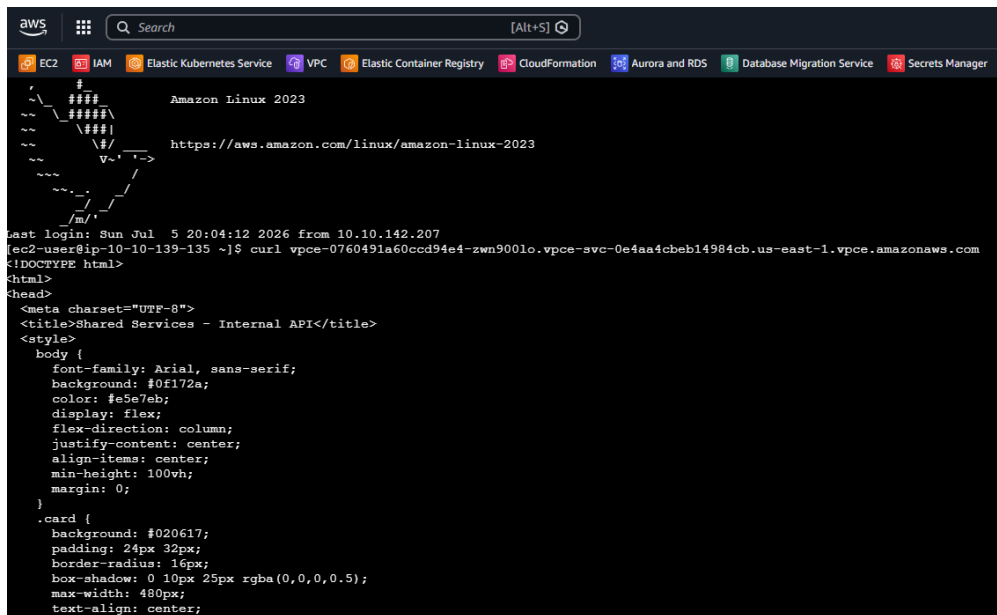
Verifying the Deployment

From outside AWS entirely, I tried to reach the internal NLB's DNS name directly from my laptop:

```
PS C:\Users\Dwayne> curl shared-services-nlb-7889b56ce55ae0b1.elb.us-east-1.amazonaws.com
curl : The remote name could not be resolved:
'shared-services-nlb-7889b56ce55ae0b1.elb.us-east-1.amazonaws.com'
```

This DNS resolution failure is the expected result for an internal NLB — its hostname simply doesn't exist in public DNS, so there isn't even a record to attempt a connection against. That's actually a stronger isolation signal than a connection timeout would have been.

From inside an approved VPC — after resolving the client IP preservation issue above — curling the Interface Endpoint's private DNS name from payments-test-instance returned the app's HTML successfully, confirming traffic flowed the full path entirely inside AWS's private network:



```
aws
Search [Alt+S]
EC2 IAM Elastic Kubernetes Service VPC Elastic Container Registry CloudFormation Aurora and RDS Database Migration Service Secrets Manager

Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Sun Jul 5 20:04:12 2026 from 10.10.142.207
[ec2-user@ip-10-10-139-135 ~]$ curl vpce-0760491a60ccd94e4-zwn900lo.vpce-svc-0efaa4cbeb14984cb.us-east-1.vpce.amazonaws.com
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Shared Services - Internal API</title>
<style>
<body {
font-family: Arial, sans-serif;
background: #0f172a;
color: #e5e7eb;
display: flex;
flex-direction: column;
justify-content: center;
align-items: center;
min-height: 100vh;
margin: 0;
}
.card {
background: #020617;
padding: 24px 32px;
border-radius: 16px;
box-shadow: 0 10px 25px rgba(0,0,0,0.5);
max-width: 480px;
text-align: center;

```

Successful curl from payments-test-instance to the Interface Endpoint's private DNS name — full HTML returned from the Shared Services app.

```
Payments VPC -> Interface Endpoint -> PrivateLink -> Internal NLB -> Target Group ->
Shared Services EC2
```


Together, these two tests proved the goal of the project: reachable only from approved VPCs via PrivateLink, and unreachable from anywhere else — including the public internet.

Project Reflection

This build took a bit longer than expected because of the client IP preservation issue, but that turned out to be the most instructive part of the whole project — it wasn't something I would have anticipated from the architecture diagram alone, and diagnosing a silent security-group drop versus an active refusal is a distinction I'll carry into future troubleshooting. Compared to the AMI-based version of this project, using a NAT Gateway made setup noticeably faster (no builder instance, no image bake/verify/deregister cycle), at the cost of an ongoing NAT Gateway hourly charge that the AMI version avoids entirely.

Next time, I'd like to explore automating this build with Terraform or CloudFormation, and extending it to a second Availability Zone for high availability on the NLB and NAT Gateway.